

RAG & Multi-Agent Systems

Make AI Answer From Your Documents · Build Teams of Specialised Agents

■ Days	23 & 24 (Back-to-Back)
■ Total Time	12 Hours (6 hrs × 2 days)
■ Difficulty	Moderate ■■■
■ Language	Python 3.10+
■ Tools	LangChain · ChromaDB · Ollama · CrewAI · DuckDuckGo
■ Cost	100% Free — no paid API required

Complete guide with working code, real examples, interview Q&A, and zero-cost setup.

Table of Contents

DAY 23	RAG – Make AI Answer From Your Own Documents
1	The Problem with LLMs (Hallucination)
2	RAG Architecture – How It Works
3	The 6-Step RAG Pipeline (with visuals)
4	Setting Up Free Tools (Ollama + ChromaDB)
5	Building Your First RAG System – Full Code
6	Lab: College Prospectus Q&A; Bot
7	Embedding Deep Dive – What Are Vectors?
8	Day 23 Interview Questions & Answers
DAY 24	AI Agents with Tools & Multi-Agent Intro
9	Giving Agents More Tools (Web Search, CSV, Code)
10	Multi-Turn Memory – Agents That Remember
11	Introduction to CrewAI – Teams of Agents
12	Lab: Personal Research Assistant (Researcher + Writer)
13	Day 24 Interview Questions & Answers
14	Combined Cheatsheet – Days 23 & 24

DAY 23

RAG – Make AI Answer From Your Own Documents

Section 1 · The Problem LLMs Have — Hallucination

Large Language Models like GPT-4, Claude, and Mistral are trained on internet data up to a certain date. They know nothing about your private documents, your company's internal policies, or anything that happened after their training cutoff. When asked about these topics, they don't say 'I don't know' — they confidently make up plausible-sounding but completely wrong answers. This is called hallucination.

Scenario	What LLM does WITHOUT RAG	What LLM does WITH RAG
'What is our leave policy?'	Invents a policy based on typical HR norms — completely wrong	Reads your HR PDF and quotes the exact policy
'What is the fee for BTech CSE?'	Guesses based on average college fees in India — may be off	Reads your college prospectus and gives the exact figure
'What happened in last week's board meeting?'	Has no data — confabulates something plausible	Reads the meeting minutes PDF and summarises accurately
'What are the API rate limits?'	Gives generic rate limits — may be outdated	Reads your internal API docs and gives exact limits

■ **RAG = Retrieval-Augmented Generation.** We augment the LLM's generation with retrieved facts from **YOUR** documents.

Why Not Just Fine-Tune the Model?

Approach	Cost	Time	Update frequency	Best for
Fine-tuning	High (\$\$\$)	Days–weeks	Retrain needed	Changing model behaviour/style
RAG	Near-zero	Minutes	Instant update	Answering from your documents
Prompt stuffing	Free	Instant	Instant	Small docs (< 10 pages)

Section 2 · RAG Architecture – How It Works

RAG has two distinct phases that you must understand clearly for interviews:

Phase	When it runs	What happens	Tools used
INDEXING (Offline)	Once, before queries start	Load docs → Split → Embed → Store in vector DB	DocumentLoader, TextSplitter, Embedding Model, ChromaDB
RETRIEVAL (Online)	Every time user asks a question	Embed query → Search DB → Get top-k chunks → Feed to LLM	Embedding Model, ChromaDB, LLM

The Embedding Concept — Critical for Interviews

An embedding is a list of numbers (a vector) that represents the meaning of text. Similar meanings → similar vectors → close together in vector space. This is how RAG finds relevant chunks even when the wording is different.

■ python

```
# Conceptual example – what an embedding looks like
# The model converts text into a list of ~1536 numbers
text_1 = 'What is the hostel fee?'
embed_1 = [0.12, -0.34, 0.89, 0.02, ..., -0.67] # 1536 numbers
text_2 = 'How much does dormitory accommodation cost?'
embed_2 = [0.11, -0.33, 0.91, 0.03, ..., -0.65] # very similar!
text_3 = 'What is the capital of France?'
embed_3 = [0.78, 0.22, -0.41, 0.90, ..., 0.12] # very different!
# Cosine similarity: embed_1 vs embed_2 = 0.98 (very close!)
# Cosine similarity: embed_1 vs embed_3 = 0.03 (unrelated!)
# RAG returns embed_2's chunk – even though wording is different!
```

Section 3 · The 6-Step RAG Pipeline

1

LOAD — Read your documents into Python

Support PDF, Word (.docx), TXT, HTML, CSV, web pages. LangChain has 50+ loaders. PyPDFLoader is most common for PDFs.

2

SPLIT — Chop documents into small chunks

LLMs have limited context. Split into ~500-1000 char chunks with 100-char overlap so no sentence is cut in the middle.

3

EMBED — Convert each chunk into a number vector

Use an embedding model (all-MiniLM-L6-v2 for free, or text-embedding-3-small for OpenAI). Each chunk becomes a point in high-dimensional space.

4

STORE — Save vectors in a vector database

ChromaDB (free, local), FAISS (Meta, free), Pinecone (cloud, paid). The DB enables fast similarity search across thousands of chunks.

5

RETRIEVE — Find the most relevant chunks for the user's question

The question is also embedded. The DB returns top-k (e.g., 3-5) most similar chunks using cosine similarity.

6

GENERATE — Feed chunks + question to LLM → accurate answer

The LLM receives: [system: 'Answer only from the context below'] + chunks + question. It generates a grounded, accurate answer using only your documents.

Section 4 · Setting Up Free Tools (Ollama + ChromaDB)

Why Free Tools? Ollama runs LLMs 100% locally on your laptop — no API key, no cost.

Install Ollama (runs LLMs locally)

■ bash

```
# Step 1: Download Ollama
# Visit: https://ollama.ai → Download for your OS (Windows/Mac/Linux)

# Step 2: Pull a model (choose based on your RAM)
ollama pull mistral # 4GB RAM minimum – good quality
ollama pull gemma:2b # 2GB RAM minimum – lighter, fast
ollama pull llama3.2:3b # 3GB RAM – Meta's latest small model

# Step 3: Test it works
ollama run mistral 'What is RAG in AI?'

# Step 4: Check it's serving on localhost:11434
curl http://localhost:11434/api/tags
```

Install Python Libraries

■ bash

```
# Core RAG libraries
pip install langchain langchain-community langchain-ollama

# Vector database (free, runs locally)
pip install chromadb

# PDF loading
pip install pypdf

# Free embedding model (runs locally, no API needed)
pip install sentence-transformers

# Verify everything works
python -c "import chromadb; import langchain; print('All good!')"
```

Choosing the Right Embedding Model

Model	Size	Speed	Quality	Cost	Use for
all-MiniLM-L6-v2	90 MB	Very fast	Good	Free	Students, local dev
all-mpnet-base-v2	420 MB	Medium	Very good	Free	Production on laptop
text-embedding-3-small	API	Fast	Excellent	\$0.02/M	OpenAI users
nomic-embed-text	274 MB	Fast	Excellent	Free	Best free option

Section 5 · Building Your First RAG System – Complete Code

This is the complete, working RAG system using 100% free tools. Copy, run, and study each part carefully.

Part A — Index Your Documents (Run Once)

[illegible]

```
embedding_model = SentenceTransformerEmbeddings(
    model_name='all-MiniLM-L6-v2'
)

print('Creating embeddings... (first run downloads the model)')

# Create ChromaDB vector store
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embedding_model,
    persist_directory=persist_dir
)

print(f'Saved {vectorstore._collection.count()} vectors to {persist_dir}')
return vectorstore

# ■■■ RUN INDEXING ■■■
```

```
if __name__ == '__main__':
    # Replace with your actual file path
    FILE = 'college_prospectus.pdf' # or 'hr_policy.txt'

    docs = load_documents(FILE)
    chunks = split_documents(docs)
    vs = create_vector_store(chunks)

    print('\nIndexing complete! Now run rag_query.py to ask questions.')
```

Part B — Query Your Documents (Run Every Time)

- python

[illegible]

Section 6 · Lab — College Prospectus Q&A; Bot

In this lab, you build a bot that answers questions about a college prospectus PDF. Since you may not have a real prospectus, we provide a sample text file to use instead.

Create a Sample Prospectus Text File (if no PDF)

■ python

```
# create_sample_prospectus.py
sample_text = """
LAKSHMI ENGINEERING COLLEGE — PROSPECTUS 2024-25

ABOUT THE COLLEGE
Lakshmi Engineering College was established in 2001 and is affiliated to
Osmania University, Hyderabad. It is NAAC A+ accredited.

COURSES OFFERED
- B.Tech Computer Science Engineering (CSE): 120 seats
- B.Tech Electronics and Communication (ECE): 60 seats
- B.Tech Mechanical Engineering (ME): 60 seats
- M.Tech Artificial Intelligence: 30 seats

FEE STRUCTURE (Annual)
- B.Tech CSE: Rs. 85,000 per year
- B.Tech ECE: Rs. 80,000 per year
- B.Tech ME: Rs. 75,000 per year
- M.Tech AI: Rs. 65,000 per year

HOSTEL FACILITIES
- Boys Hostel: Rs. 45,000 per year (food included)
- Girls Hostel: Rs. 48,000 per year (food included)
- Rooms: Double-sharing, AC rooms available for Rs. 10,000 extra

ADMISSION CRITERIA
- B.Tech: TS EAMCET rank required. Minimum 60% in Class 12 (PCM).
- M.Tech: GATE score mandatory. Minimum 60% in B.Tech.

PLACEMENT RECORD (2023-24)
- Placement rate: 92%
- Highest package: Rs. 42 LPA (Microsoft)
- Average package: Rs. 8.5 LPA
- Top recruiters: TCS, Infosys, Wipro, Microsoft, Amazon, Google

CONTACT
Email: admissions@lec.edu.in | Phone: 040-23456789
"""

with open('prospectus.txt', 'w') as f:
    f.write(sample_text)
print('Sample prospectus created: prospectus.txt')
```

Run the Full Pipeline

■ bash

```
# Step 1: Create sample data
python create_sample_prospectus.py

# Step 2: Index it (run once)
# In rag_indexer.py, change FILE = 'prospectus.txt'
python rag_indexer.py

# Output:
```

```
# Loaded 1 pages/sections from prospectus.txt
# Split into 8 chunks
# Saved 8 vectors to ./chroma_db
# Step 3: Ask questions!
python rag_query.py
```

Expected Sample Session

■ text

RAG Q&A Bot – Ask questions about your document!

Your question: What is the fee for BTech CSE?

Retrieved 3 chunks from DB:

[1] Page ? – FEE STRUCTURE (Annual) - B.Tech CSE: Rs. 85,000 per year...

[2] Page ? – COURSES OFFERED - B.Tech Computer Science Engineering (CSE)...

[3] Page ? – HOSTEL FACILITIES - Boys Hostel: Rs. 45,000 per year...

A: The annual fee for B.Tech Computer Science Engineering (CSE) is
Rs. 85,000 per year.

Your question: What is the highest salary package?

A: The highest package in 2023-24 was Rs. 42 LPA, offered by Microsoft.

Your question: Tell me about the prime minister of India.

A: I could not find this information in the provided documents.

(This demonstrates that RAG does NOT hallucinate!)

Section 7 · Embedding Deep Dive – What Are Vectors?

This section is essential for interview success. Interviewers love asking about embeddings.

■ python

```
# embedding_demo.py – Run this to SEE embeddings in action
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

# Test sentences
sentences = [
    'What is the hostel fee?',
    'How much does dormitory accommodation cost?',
    'Tell me about the placement record.',
    'What is the capital of France?',
]

# Generate embeddings
embeddings = model.encode(sentences)
print(f'Embedding shape: {embeddings.shape}') # (4, 384)
print(f'Each embedding has 384 numbers')

# Compute cosine similarity between all pairs
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
print('\nSimilarity scores (1.0 = identical, 0.0 = unrelated):')
for i in range(len(sentences)):
    for j in range(i+1, len(sentences)):
        sim = cosine_similarity(embeddings[i], embeddings[j])
        print(f' [{i+1}] vs [{j+1}]: {sim:.3f} – "{sentences[i][:30]}..."')

# Expected output:
# [1] vs [2]: 0.847 – very similar (same meaning, different words)
# [1] vs [3]: 0.231 – somewhat related (both about college)
# [1] vs [4]: 0.043 – unrelated (hostel vs France capital)
```

Q1. What is RAG? Why do we need it?

RAG (Retrieval-Augmented Generation) is a technique that grounds LLM responses in real documents by first retrieving relevant passages and then generating answers based on them. We need it because LLMs hallucinate when asked about private, recent, or domain-specific information they weren't trained on. RAG gives the LLM real facts to reference, eliminating hallucination for document-based questions.

Q2. Explain the 6 steps of a RAG pipeline.

1) Load: read documents (PDF, TXT, etc.) using document loaders. 2) Split: divide into ~500-char overlapping chunks using RecursiveCharacterTextSplitter. 3) Embed: convert each chunk into a vector using an embedding model. 4) Store: save vectors in a vector database like ChromaDB or FAISS. 5) Retrieve: embed the user's question and find top-k similar chunks. 6) Generate: pass the question + retrieved chunks to the LLM for an accurate answer.

Q3. What is an embedding / vector? Why are they used in RAG?

An embedding is a list of floating-point numbers (e.g., 384 or 1536) that represents the semantic meaning of text. Similar meaning → similar vectors. They are used in RAG because they enable semantic search: finding relevant text even when exact keywords don't match. Example: 'hostel cost' and 'dormitory fees' produce similar embeddings and will be matched correctly.

Q4. What is chunk_overlap and why is it important?

chunk_overlap is the number of characters shared between adjacent chunks when splitting. Without overlap, a sentence spanning the boundary of two chunks could be split in half, losing critical context. With overlap (e.g., 50 chars), both chunks contain the complete sentence. This prevents information loss at chunk boundaries.

Q5. What is ChromaDB? How is it different from a regular SQL database?

ChromaDB is a vector database optimised for storing and searching embeddings. A regular SQL database searches by exact matching (WHERE name = 'Priya'). ChromaDB searches by semantic similarity (cosine distance between vectors), returning the most meaning-similar entries even without exact keyword matches. It's designed specifically for AI/ML workloads with embedding search.

Q6. What is the difference between RAG and fine-tuning?

Fine-tuning re-trains the model on your data, changing its weights permanently. It's expensive (\$\$\$ GPU compute), takes days, and requires retraining for every update. RAG doesn't change the model at all — it just gives the model documents to read at query time. RAG is instant to update (just re-index documents), near-zero cost, and more transparent (you can see which chunks were used). Use fine-tuning to change behaviour/style; use RAG to answer questions from specific documents.

DAY 24

AI Agents with Tools & Multi-Agent Introduction

Section 9 · Giving Agents More Tools

A single-tool agent is limited. Real agents need a toolbox. Today we add web search, a CSV reader, and a code executor — all free.

Tool 1 — DuckDuckGo Web Search (100% Free)

■ bash

```
# web_search_tool.py
pip install duckduckgo-search langchain-community
```

■ python

```
from langchain_community.tools import DuckDuckGoSearchRun
from langchain.tools import Tool

# Initialize DuckDuckGo search (no API key needed)
ddg_search = DuckDuckGoSearchRun()

web_search_tool = Tool(
    name='WebSearch',
    func=ddg_search.run,
    description=(
        'Search the internet for current information. '
        'Use for recent news, current events, live data, '
        'or any topic that may have changed recently. '
        'Input: a search query string.'
    )
)

# Test it directly
result = ddg_search.run('India GDP 2024')
print(result[:300])
```

Tool 2 — CSV Query Tool

■ python

```
# csv_query_tool.py
import pandas as pd
from langchain.tools import Tool

# Sample CSV – save as 'sales_data.csv'
# product,region,sales,month
# Laptop,North,45000,Jan
# Mobile,South,32000,Jan
# Laptop,South,52000,Feb

def query_csv(query: str) -> str:
    """
    Query a CSV file using natural language.
```

```

Supports: total, average, max, min, count, filter.
Format: 'column|operation' or just describe what you want.
"""
try:
df = pd.read_csv('sales_data.csv')
q = query.lower()
if 'total sales' in q or 'sum of sales' in q:
total = df['sales'].sum()
return f'Total sales across all records: Rs.{total:,}'
elif 'average' in q or 'mean' in q:
avg = df['sales'].mean()
return f'Average sales per record: Rs.{avg:,.2f}'
elif 'highest' in q or 'maximum' in q or 'top' in q:
idx = df['sales'].idxmax()
row = df.loc[idx]
return f'Highest: {row["product"]} in {row["region"]} = Rs.{row["sales"]:,}'
elif 'by region' in q or 'region-wise' in q:
result = df.groupby('region')['sales'].sum()
return 'Sales by region:\n' + result.to_string()
elif 'by product' in q:
result = df.groupby('product')['sales'].sum()
return 'Sales by product:\n' + result.to_string()
else:
return f'Available columns: {list(df.columns)}. Rows: {len(df)}'
except Exception as e:
return f'CSV error: {str(e)}'
csv_tool = Tool(
name='CSVQuery',
func=query_csv,
description=(
'Query the sales CSV database for business insights. '
'Use for total sales, average, highest, by region, by product queries.'
)
)

```

Tool 3 — Python Code Executor

■ python

```

import subprocess, sys
from langchain.tools import Tool

def run_python_code(code: str) -> str:
    """
    Execute Python code and return the output.
    Use for complex calculations, data processing, or algorithms.
    Input: valid Python code as a string.
    """
    try:
        # Run in a subprocess for safety
        result = subprocess.run(
            [sys.executable, '-c', code],
            capture_output=True, text=True, timeout=10
        )
    
```

```

if result.returncode == 0:
    return result.stdout or 'Code ran successfully (no output)'
else:
    return f'Error: {result.stderr}'
except subprocess.TimeoutExpired:
    return 'Error: Code took too long (>10 seconds)'
except Exception as e:
    return f'Execution error: {str(e)}'

code_tool = Tool(
    name='PythonExecutor',
    func=run_python_code,
    description=(
        'Execute Python code for complex calculations or data processing. '
        'Input: Python code as a string. '
        'Example: "import math; print(math.factorial(10))"'
    )
)

```

Combine All Tools in One Agent

■ python

```

# multi_tool_agent.py
from langchain_openai import ChatOpenAI # or ChatOllama for free
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub

# All tools together
tools = [web_search_tool, csv_tool, code_tool, calculator_tool]
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
prompt = hub.pull('hwchase17/react')
agent = create_react_agent(llm=llm, tools=tools, prompt=prompt)
exe = AgentExecutor(agent=agent, tools=tools, verbose=True,
max_iterations=8, handle_parsing_errors=True)

# Test with questions that require different tools
questions = [
    'What is 15% compound interest on Rs.1 lakh for 3 years?', # code tool
    'What is the total sales in our CSV by region?', # csv tool
    'What are the latest AI news from India today?', # web search
]

for q in questions:
    print(f'\nQ: {q}')
    r = exe.invoke({'input': q})
    print(f'A: {r["output"]}')

```

Section 10 · Multi-Turn Memory – Agents That Remember

By default, every agent call is stateless — the agent forgets everything from the previous turn. Memory fixes this.

Memory Type	What it stores	Capacity	Best for
ConversationBufferMemory	Full history verbatim	Limited by context window	Short chats (< 20 turns)
ConversationSummaryMemory	Summary of old turns	Unlimited	Long chats
ConversationBufferWindowMemory	Last N turns only	Fixed window	Sliding window chats
VectorStoreRetrieverMemory	Semantically relevant old turns	Unlimited	Long-term personal context

Complete Memory-Enabled Chatbot

- python

[illegible]

```
'Can you summarise what we have discussed so far?',  
]  
for turn in turns:  
    print(f'\nUser: {turn}')  
    result = exe.invoke({'input': turn})  
    print(f'Bot: {result["output"]}')
```

Expected output for turn 3:
Bot: Your name is Priya and you are studying AI at VIT Hyderabad.

Section 11 · Introduction to CrewAI – Teams of Specialised Agents

What happens when one agent isn't enough? You create a team. CrewAI lets you define specialised agents with different roles and let them collaborate — just like a real human team.

Why Multi-Agent? The Division of Labour Principle

Approach	Problem	Solution with CrewAI
Single agent does research + writing	Tries to do too much; quality suffers; hits context limits	Researcher agent focuses ONLY on gathering facts
Single agent evaluates its own output	Can't objectively critique itself	Critic agent reviews and improves Researcher's output
Single agent handles all domains	No specialisation; generic outputs	Domain-specific agents with custom tools and prompts

Install CrewAI

```
pip install crewai crewai-tools duckduckgo-search
```

Complete 2-Agent Research Crew

```
# research_crew.py - A Researcher + Writer team
from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool # or use DuckDuckGo
from langchain_openai import ChatOpenAI
from langchain_community.tools import DuckDuckGoSearchRun
from crewai.tools import BaseTool
from pydantic import Field

# ■■■ CUSTOM DUCKDUCKGO TOOL FOR CREWAI ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
class DuckDuckGoTool(BaseTool):
    name: str = 'DuckDuckGo Search'
    description: str = 'Search the internet for current information on any topic.'
    search: DuckDuckGoSearchRun = Field(default_factory=DuckDuckGoSearchRun)

    def _run(self, query: str) -> str:
        return self.search.run(query)

search_tool = DuckDuckGoTool()

# ■■■ DEFINE LLM ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.3)

# ■■■ AGENT 1: THE RESEARCHER ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
researcher = Agent(
    role='Senior Research Analyst',
    goal='Find comprehensive, accurate, and up-to-date information on {topic}',
    backstory=(
        'You are an expert researcher with 10 years of experience. '
        'You know how to search effectively, evaluate sources, '
        'and extract the most important facts and statistics. '
        'You always cite where information comes from.'
    ),
    tools=[search_tool],
```

[illegible]

[illegible]

■ CrewAI Key Concepts: *role* = agent's job title | *goal* = what it optimises for | *backstory* = personality and expertise | *context*=[task] = output feeding chain | *Process.sequential* = tasks run in order | *Process.hierarchical* = manager coordinates agents.

Section 12 · Lab — Personal Research Assistant (Full System)

This lab combines EVERYTHING from Days 23 and 24: RAG + Multi-Tool Agent + CrewAI. Build a complete research assistant that takes a topic and produces a polished report.

System Architecture

Component	Role	Tools	Tech
Researcher Agent	Search the web for current info	DuckDuckGo Search	CrewAI + LangChain
RAG Agent	Query your own documents	ChromaDB Retriever	LangChain RAG
Writer Agent	Synthesise and write the report	None (pure writing)	CrewAI
Critic Agent	Review and improve the report	None (pure review)	CrewAI

Free Alternative: Groq + CrewAI (Zero Cost)

■ bash

```
# Free alternative using Groq (Llama 3.1 70B – very fast, free tier)
pip install groq langchain-groq
```

■ python

```
from langchain_groq import ChatGroq
import os

# Get free API key at: console.groq.com
llm = ChatGroq(
    model='llama-3.1-70b-versatile',
    temperature=0.3,
    api_key=os.getenv('GROQ_API_KEY')
)

# Use this llm in place of ChatOpenAI in ALL previous code examples!
# CrewAI, LangChain agents, RAG chains – all work with Groq

# Quick test
response = llm.invoke('What is RAG in AI? Answer in 2 sentences.')
print(response.content)
```

Complete Interactive Research Assistant

■ python

```
# research_assistant.py – Full CLI research assistant
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI # or ChatGroq for free
# ... (import search_tool from earlier)

def run_research_assistant(topic: str, depth: str = 'detailed'):
    """
    Run a full research pipeline on any topic.
    depth: 'quick' (1 search), 'detailed' (3 searches), 'deep' (5 searches)
    """
    llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.3)
```


Q1. What is the difference between a single agent and a multi-agent system?

A single agent uses one LLM that tries to do everything — research, write, critique, and more. Quality degrades when too many tasks are combined. A multi-agent system assigns specialised roles to different agents: a Researcher with search tools, a Writer focused on language, a Critic for quality review. Each agent excels at its specific task. Multi-agent systems handle complex, long-horizon tasks better.

Q2. What is ConversationBufferMemory? When would you use a different type?

ConversationBufferMemory stores the complete conversation verbatim in the context. Use it for short chats (< 20 turns). For very long conversations, use ConversationSummaryMemory (compresses old turns into summaries), ConversationBufferWindowMemory (keeps only last N turns), or VectorStoreRetrieverMemory (retrieves semantically relevant past messages). Choice depends on conversation length and which past context needs to be retained.

Q3. Explain the role of the 'description' field in a LangChain Tool.

The description is the most critical part of a tool. The LLM reads it to decide WHEN and HOW to call the tool. A bad description = agent calling wrong tools. Good descriptions specify: what the tool does, when to use it, the exact input format, and an example. Bad: 'Searches the web'. Good: 'Search the internet for current news and recent events. Input: a specific search query. Example: India stock market today.'

Q4. What is CrewAI? What are its key components?

CrewAI is a framework for building and orchestrating teams of AI agents. Key components: Agent (an LLM with a role, goal, backstory, and tools), Task (a specific assignment with description and expected output, assigned to one agent), Crew (the team: list of agents + list of tasks + orchestration process), Process (sequential = tasks run in order; hierarchical = manager delegates to agents).

Q5. What is Process.sequential vs Process.hierarchical in CrewAI?

Sequential: tasks run one after another in defined order. Output of task N becomes context for task N+1. Simple and predictable — good for research → writing pipelines. Hierarchical: a manager agent reads all tasks, delegates them to workers, and synthesises the results. More flexible but less predictable. Use sequential for known pipelines; hierarchical for dynamic, open-ended workflows.

Q6. How would you combine RAG and Agents in one system?

Build a RAG retriever and wrap it as an agent tool: Tool(name='DocumentSearch', func=rag_chain.invoke, description='Search internal documents'). Add it alongside web_search and calculator tools. The agent then decides: for questions about your documents, it calls DocumentSearch (RAG); for current events, it calls WebSearch; for calculations, the calculator. This hybrid system handles both private knowledge and public information.

Q7. What is DuckDuckGo search tool and why use it over Google/Bing?

DuckDuckGo Search provides a free, no-API-key search that LangChain wraps as DuckDuckGoSearchRun. Google and Bing require paid API keys (Serper, Bing Search API). For learning and prototyping, DuckDuckGo is ideal: zero cost, instant setup, no rate limits for moderate use. For production, Serper (Google-backed) or Tavily API are preferred for better results.

Q8. What is the context parameter in CrewAI Tasks?

context=[task1, task2] tells CrewAI that this task depends on the output of the listed tasks. The framework automatically passes the output of context tasks into the current task's execution. This enables chaining: the Writer task with context=[research_task] automatically receives the Researcher's findings. Without context, agents don't share information between tasks.

RAG Quick Reference (Day 23)

■ python

```
# COMPLETE RAG IN 15 LINES
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.embeddings import SentenceTransformerEmbeddings
from langchain_community.vectorstores import Chroma
from langchain_ollama import ChatOllama
from langchain.chains import RetrievalQA

docs = PyPDFLoader('your_doc.pdf').load()
chunks = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50).split_documents(docs)
emb = SentenceTransformerEmbeddings(model_name='all-MiniLM-L6-v2')
db = Chroma.from_documents(chunks, emb, persist_directory='./db')
llm = ChatOllama(model='mistral')
qa = RetrievalQA.from_chain_type(llm=llm, retriever=db.as_retriever(search_kwargs={'k':3}))
answer = qa.invoke({'query': 'Your question here'})
print(answer['result'])
```

Agent + Tools Quick Reference (Day 24)

■ python

```
# AGENT WITH TOOLS IN 12 LINES
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from langchain.tools import Tool
from langchain import hub

tools = [Tool(name='T', func=my_func, description='What this tool does.')]
llm = ChatOpenAI(model='gpt-4o-mini', temperature=0)
agent = create_react_agent(llm, tools, hub.pull('hwchase17/react'))
exe = AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterations=5)
print(exe.invoke({'input': 'Your question'})['output'])
```

CrewAI Quick Reference (Day 24)

■ python

```
# 2-AGENT CREW IN 20 LINES
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model='gpt-4o-mini')

a1 = Agent(role='Researcher', goal='Gather facts on {topic}',
backstory='Expert analyst', tools=[search_tool], llm=llm)
a2 = Agent(role='Writer', goal='Write report on {topic}',
backstory='Expert writer', tools=[], llm=llm)

t1 = Task(description='Research {topic} thoroughly.',
expected_output='Research brief', agent=a1)
t2 = Task(description='Write a structured report on {topic}.',
```

```
expected_output='Markdown report', agent=a2, context=[t1])

crew = Crew(agents=[a1,a2], tasks=[t1,t2], process=Process.sequential)
result = crew.kickoff(inputs={'topic': 'AI in India'})
print(result.raw)
```

Key Terms — Days 23 & 24

RAG	Retrieval-Augmented Generation — ground LLM in your docs
Embedding	List of numbers representing text meaning — enables semantic search
Vector DB	Database optimised for similarity search on embeddings (ChromaDB, FAISS)
Chunk	Small piece of document (~500 chars) — the unit of retrieval
chunk_overlap	Shared characters between chunks — prevents context loss at boundaries
Hallucination	LLM confidently making up wrong information — RAG prevents this
Retriever	Component that finds top-k relevant chunks for a query
CrewAI	Framework for multi-agent teams — Agents, Tasks, Crew, Process
Agent.role	Job title of the agent — affects how it reasons and behaves
Task.context	List of tasks whose outputs are passed to this task — enables chaining
Process.sequential	Tasks run in defined order — simple, predictable pipelines
DuckDuckGo	Free web search tool — no API key required, great for learning
memory_key	Name of the variable holding chat history in the prompt template
max_iterations	Safety limit on agent loop iterations — prevents infinite loops
allow_delegation	Whether a CrewAI agent can hand off tasks to another agent

Free Tools Summary

Tool	Purpose	Cost	Install
Ollama + Mistral	Local LLM (no API needed)	Free	ollama.ai
ChromaDB	Local vector database	Free	pip install chromadb
SentenceTransformers	Local embedding model	Free	pip install sentence-transformers
DuckDuckGo Search	Web search, no API key	Free	pip install duckduckgo-search
Groq API (Llama 70B)	Fast cloud LLM, free tier	Free*	console.groq.com
CrewAI	Multi-agent framework	Free	pip install crewai

Days 23 & 24 Complete — RAG + Multi-Agent Systems Mastered! ■

Next: Day 25 — Fine-Tuning LLMs & PEFT Techniques